

How to connect the control node and manage node :

I tried to install the ansible by directly through but it through me an error

Sudo install ansible -y

Instead of that I tried it to download it through the pip of python :

- `sudo dnf install python3-pip -y`
- `pip3 install --user ansible`
- `ansible --version`(to see wheather the ansible had been installed or not)

If you get `command not found`, add pip's user bin directory to PATH:

```
bash

export PATH=$PATH:~/.local/bin
echo 'export PATH=$PATH:~/.local/bin' >> ~/.bashrc
source ~/.bashrc
ansible --version
```

Then we need to generate an ssh key pair on the control node :

```
prodevans@localhost:~$ ssh-keygen -t rsa -b 4096
Generating public/private rsa key pair.
Enter file in which to save the key (/home/prodevans/.ssh/id_rsa):
Created directory '/home/prodevans/.ssh'.
Enter passphrase for "/home/prodevans/.ssh/id_rsa" (empty for no passphrase):
```

Why we should use the key phrase :

Key Reasons for Using a Passphrase

○ Protection Against Key Theft:

If an unauthorized user gets access to your Ansible control node (or reads a backup of it), they cannot simply copy your private key and use it to log into all your managed servers. Without the passphrase, the private key file remains encrypted and useless.

○ Defense in Depth:

Ansible often manages sensitive infrastructure with elevated (`root` / `sudo`) access. A passphrase acts as a second layer of security, ensuring that having local file access on the control node isn't automatically enough to compromise your entire fleet.

Then it will create an location of the file where the private key is stored :

```
Enter same passphrase again:
Your identification has been saved in /home/prodevans/.ssh/id_rsa
Your public key has been saved in /home/prodevans/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:SBj7QSJJGj0m+h9/m57rqlFBfjsnksOWzkfmXgoJ4CA prodevans@localhost.localdomain
The key's randomart image is:
+---[RSA 4096]-----+
|.oo.o o
|.o=. O
|Eo..o = .
|oo . + B .
|.. . @ S .
|. * B +
|..o= o .
|...+. =
|...=Xo
+-----[SHA256]-----+
prodevans@localhost:~$
```

Then once the private and public key is generated then we need to do on control node and the manage nodes:

```
root@192.168.50.73's password:
prodevans@localhost:~$ ssh-copy-id manage_node1@192.168.50.73
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/prodevans/.ssh/id_rsa.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new key
manage_node1@192.168.50.73's password:

Number of key(s) added: 1

Now try logging into the machine, with: "ssh 'manage_node1@192.168.50.73'"
and check to make sure that only the key(s) you wanted were added.

prodevans@localhost:~$
```

This succeeded — number of key(s) added: 1. So manage_node1 is a regular user account

prodevans@lo
/usr/bin/ssh
/usr/bin/ssh
/usr/bin/ssh
manage_node1
Number of ke
Now try logg
and check to
prodevans@lo
Snipping Too

To cross verify wheather it worked or not on the control node type the manage node ssh:

```
prodevans@localhost:~$ ssh manage_node1@192.168.50.73
Web console: https://localhost:9090/ or https://192.168.50.73:9090/

Register this system with Red Hat Lightspeed: rhc connect

Example:
# rhc connect --activation-key <key> --organization <org>

The rhc client and Red Hat Lightspeed will enable analytics and additional
management capabilities on your system.
View your connected systems at https://console.redhat.com/insights

You can learn more about how to register your system
using rhc at https://red.ht/registration
Last login: Mon Jul 20 15:52:43 2026 from 192.168.50.12
manage_node1@localhost:~$ exit
logout
Connection to 192.168.50.73 closed.
prodevans@localhost:~$
```

[managed nodes:vars]

Then check whether on the both managed nodes whether the python is installed or not by using the “python –version”

Then we are creating an folder on the control node :

```
prodevans@localhost:~$ ls
ansible-lab Desktop Documents Downloads id_rsa.pub Music Pictures Public Templates Videos
prodevans@localhost:~$ cd ansible-lab/
prodevans@localhost:~/ansible-lab$
```

The inventory file tells to the ansible which machine exists and how to connect to them:

```
GNU nano 8.1 inventory.ini
[managed_nodes]
node1 ansible_host=192.168.50.73 ansible_user=manage_node1
node2 ansible_host=192.168.50.136 ansible_user=manage-node
```

Create an ansible.conf file:

```
This file sets default behavior so you don't need extra flags every time.

bash
nano ansible.cfg

ini
[defaults]
inventory = ./inventory.ini
host_key_checking = False

Explanation:
• inventory = ./inventory.ini → Ansible automatically uses this file
-i inventory.ini in every command
• host_key_checking = False → skips the "Are you sure you want to c
(yes/no)?" prompt that normally appears the first time you SSH into
a homelab; in production you'd want this on for security.

Save and exit.
```

Once all the steps are done ,then cross verify this from your control node:

```
prodevans@localhost:~$ ls
ansible-lab Desktop Documents Downloads id_rsa.pub Music Pictures Public Templates Videos
prodevans@localhost:~$ cd ansible-lab/
prodevans@localhost:~/ansible-lab$ ansible node1 -m ping
[WARNING]: Host 'node1' is using the discovered Python interpreter at '/usr/bin/python3.12', but future installation of
another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-co
re/2.21/reference_appendices/interpreter_discovery.html for more information.
node1 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.12"
  },
  "changed": false,
  "ping": "pong"
}
prodevans@localhost:~/ansible-lab$
```

Run a real command on both managed nodes (proves shell execution works, not just the ping module):

```
manage_node1@localhost:~$ ansible managed_nodes -m command -a "uptime"
bash: ansible: command not found...
manage_node1@localhost:~$ exit
logout
Connection to 192.168.50.73 closed.
prodevans@localhost:~/ansible-lab$ cd ~/ansible-lab
ansible managed_nodes -m command -a "uptime"
[WARNING]: Host 'node2' is using the discovered Python interpreter at '/usr/bin/python3.12', but future installation of
another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-co
re/2.21/reference_appendices/interpreter_discovery.html for more information.
node2 | CHANGED | rc=0 >>
  16:30:46 up 39 min,  2 users,  load average: 0.00, 0.01, 0.01
[WARNING]: Host 'node1' is using the discovered Python interpreter at '/usr/bin/python3.12', but future installation of
another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-co
re/2.21/reference_appendices/interpreter_discovery.html for more information.
node1 | CHANGED | rc=0 >>
  16:30:47 up 40 min,  1 user,  load average: 0.03, 0.02, 0.00
```

Real world use case:

Great question — understanding the "why" behind this makes everything you just did click into place. Let's connect it to real-world scenarios.

The Core Problem Ansible Solves

Imagine you're a sysadmin/DevOps engineer managing **not 2, but 200 servers**. Without Ansible, if you needed to install a security patch on all of them, you'd have to:

```
ssh server1
```

```
sudo dnf update -y
```

```
exit
```

```
ssh server2
```

```
sudo dnf update -y
```

```
exit
```

... (repeat 198 more times)

This is slow, error-prone, and impossible to track. Ansible lets you run **one command from your control node** that does this across all 200 servers simultaneously:

```
ansible all -m dnf -a "name=* state=latest"
```

The SSH connectivity you just set up is the **foundation** that makes this possible — it's literally the pipe through which every future automation task flows.

Real-World Use Case 1: Patch Management / Updates

Scenario: Your company has 50 web servers. A critical CVE (security vulnerability) drops today and needs patching across all of them before end of day.

```
ansible webservers -m dnf -a "name=openssl state=latest" --become
```

One command, 50 servers patched, with a report showing exactly which succeeded/failed. This is the exact ping connectivity test you just did — scaled up.

Real-World Use Case 2: Configuration Consistency (Config Drift Prevention)

Scenario: You have web servers that are all supposed to have identical Nginx configs, but over months of manual SSH tweaks, they've drifted apart — one has a different timeout setting, another has an extra config file someone forgot to remove.

With Ansible, you write a **playbook** (a YAML file describing desired state) once:

```
- name: Ensure nginx config is standardized
```

```
  hosts: webservers
```

```
  tasks:
```

```
    - name: Copy standard nginx.conf
```

```
      copy:
```

```
        src: nginx.conf
```

```
        dest: /etc/nginx/nginx.conf
```

```
      notify: restart nginx
```

Run it against all servers — every single one is forced back into the exact same state. This is huge in real companies because "works on my server but not that one" bugs are extremely common and costly to debug.

Real-World Use Case 3: New Server Provisioning (Onboarding)

Scenario: Your company spins up a new VM every time a developer needs a test environment, or when scaling up for traffic. Instead of manually installing Docker, configuring users, setting firewall rules, and hardening SSH by hand each time (30+ minutes of repetitive work), you write it once as a playbook:

```
- name: Bootstrap new server
```

```
  hosts: new_servers
```

```
  tasks:
```

```
    - name: Install Docker
```

```
      dnf: name=docker state=present
```

- name: Create deploy user

user: name=deploy groups=docker

- name: Configure firewall

firewalld: service=https permanent=true state=enabled

This turns a 30-minute manual process into a 2-minute automated one — and it's **identical every time**, eliminating human error.

Real-World Use Case 4: Your Specific Homelab (Foreman/Katello context)

Since you're building out Foreman/Katello and RHEL VMs, here's how this directly applies to what you're doing:

- **Foreman/Katello** manages content (RPM repos, patches, subscriptions) — but Ansible is what actually **applies configuration** to hosts.
- In real enterprise RHEL environments, Foreman + Ansible work together: Foreman handles provisioning and patch content, Ansible (often via **Ansible Automation Platform / AWX**, Red Hat's product) handles configuration management — deploying users, applying security baselines (like CIS benchmarks), configuring DNS clients to point at your BIND server, etc.

For example, you could write a playbook that goes to every managed node and:

- name: Point all clients to homelab DNS

hosts: managed_nodes

tasks:

- name: Set DNS server

lineinfile:

path: /etc/resolv.conf

line: "nameserver 192.168.50.1"

Instead of manually editing /etc/resolv.conf on every VM you spin up, you push it out to all managed nodes in one shot.

Real-World Use Case 5: Compliance and Auditing

Scenario: Your company needs to prove to auditors that every server has disk encryption enabled, a specific password policy, and no unauthorized users. Instead of manually checking 100+ servers:

ansible all -m shell -a "cat /etc/login.defs | grep PASS_MAX_DAYS"

You get a consolidated report across your entire fleet in seconds — critical in industries like finance, healthcare, and government where compliance audits are frequent and expensive if failed.

Real-World Use Case 6: Zero-Downtime Application Deployment

Scenario: You're deploying a new version of a web app across a cluster of servers behind a load balancer. You don't want to take all servers down at once (that would cause an outage).

Ansible can do **rolling updates**:

- hosts: webservers

serial: 2 # updates 2 servers at a time, not all at once

tasks:

- name: Pull latest app version

git: repo=https://github.com/company/app.git dest=/var/www/app

- name: Restart app service

systemd: name=myapp state=restarted

This is exactly how companies like Netflix, Airbnb, and thousands of others deploy code without users ever noticing downtime.

Why the SSH Key Setup Specifically Matters at Scale

In a real company, you're not manually running ssh-copy-id for 200 servers. Instead:

- New servers get **provisioned with your control node's public key pre-installed** (via a "golden image" or PXE/kickstart automation — which ties back to your Foreman work)
- Or centralized identity management (like FreeIPA/IdM, or even Active Directory integration) handles authentication instead of individual key files
- Ansible Automation Platform (AWX/Tower) centralizes **credential management** so individual engineers don't even see the private key — it's stored securely and injected at runtime

What you did manually (generate key → copy to each host → verify) is the **conceptual foundation** every automated onboarding process is built on top of.

Summing It Up

What you did	Real-world equivalent
SSH key setup	How control planes authenticate to hundreds/thousands of servers securely, without passwords
Inventory file	A living map of your entire infrastructure — which servers exist, grouped by role (web, db, DNS, etc.)
ansible -m ping	The first health check any automation pipeline runs before deploying anything — "can I even reach this fleet?"
Ad-hoc commands (uptime, command)	Quick fleet-wide diagnostics during incidents (e.g., "is everything up? what's the load average across all servers right now?")

The 2-node setup you built is literally the exact same architecture used to manage **fleets of thousands of servers** at companies like Red Hat, Meta, and any enterprise using Infrastructure as Code — just at a smaller scale for learning. Once you're comfortable with ad-hoc commands, the natural next step is **playbooks**, which is where this really starts to shine for tasks like the config-drift and deployment examples above. Want me to walk through building your first real playbook next — maybe something practical like configuring both managed nodes to use your BIND DNS server?